

# Tema 9 - Módulos

## Contenido teórico

### Objetivos

- **Entender qué es un módulo** y por qué se usa para organizar código.
- **Crear módulos propios** con funciones, clases y constantes reutilizables.
- **Importar módulos completos**, recursos concretos y alias de forma correcta.
- **Distinguir** script ejecutable, módulo importable y paquete.
- **Conocer cómo Python localiza** módulos y cómo evitar conflictos de nombres.

### Qué es un módulo

Un módulo es un fichero `.py` que contiene código Python reutilizable: funciones, clases, constantes o variables.

- Al importar un módulo, Python ejecuta ese fichero una vez y deja disponibles sus recursos.
- **La idea principal es separar responsabilidades:** un fichero puede contener lógica de validación, otro puede tener lectura de datos y otro el script principal.

#### ¿Por qué modularizar scripts?

- En scripts pequeños todo puede estar en un único fichero, pero eso escala mal cuando el código crece.
- Los módulos permiten reutilizar funciones sin copiar y pegar, probar partes concretas y mantener mejor el proyecto.
- En TI es habitual separar operaciones como lectura de inventario, validación de puertos, generación de reportes o diagnóstico de sistema.

```
# Sin modularizar: todo mezclado
# lectura + validación + salida

# Con módulos:
# inventario.py
# validaciones.py
# reporte.py
# main.py # Código principal

# validaciones.py
# Ejemplo de módulo

def puerto_valido(puerto):
    return 1 <= puerto <= 65535

def normalizar_servicio(nombre):
    return nombre.strip().lower()
```

## Crear un módulo propio e importarlo

### Crear un módulo:

- Un módulo propio no necesita una sintaxis especial: basta con crear un fichero `.py` con código reutilizable.
- Conviene que las funciones tengan nombres claros y que el módulo agrupe operaciones relacionadas.
- El nombre del fichero será el nombre que se use en `import`, sin la extensión `.py`.

### Importar un módulo completo:

- `import nombre_modulo`: importa el módulo completo.
- Si el fichero del módulo está en el mismo directorio que el principal, Python lo localiza inmediatamente.
- `modulo.recurso`: accede a los recursos del módulo.
- Es una forma clara porque mantiene visible de dónde viene cada función o clase.
- Suele ser recomendable cuando el módulo contiene varios recursos.

```
# validaciones.py
# Ejemplo de módulo

def puerto_valido(puerto):
    return 1 <= puerto <= 65535

def normalizar_servicio(nombre):
    return nombre.strip().lower()
# -----
# main.py con importación completa
# Código principal

import validaciones

puerto = 443
if validaciones.puerto_valido(puerto):
    print("Puerto correcto")

servicio = validaciones.normalizar_servicio(" SSH ")
print(servicio)
```

## Otras formas de importación

### Importar recursos concretos:

- `from modulo import recurso`: se importa solo un nombre concreto de recurso del módulo.
- Después se puede usar directamente, sin prefijo.
- Es cómodo, pero si se importan demasiados recursos puede ser menos claro saber de dónde procede cada nombre.

### Uso de alias:

- El alias permite usar un nombre más corto o evitar conflictos. Se define con `as`.
- Se usa tanto al importar módulos completos como recursos concretos.
- Debe usarse con criterio: un alias demasiado críptico hace que el código sea menos legible.

```
# main.py con recursos concretos
from validaciones import puerto_valido

if puerto_valido(22):
```

```

    print("ssh válido")

# servicio = normalizar_servicio(" SsH ")
# NameError no está disponible con este import
# -----
# main.py con alias completo
import validaciones as val

if val.puerto_valido(443):
    print("https válido")
# -----
# main.py con alias de recurso
from validaciones import puerto_valido as valido

if valido(443):
    print("https válido")

```

## Importar todo con \* (no recomendable)

- `from modulo import *`: incorpora todos los nombres públicos del módulo al espacio actual.
- En entornos profesionales puede parecer cómodo, pero dificulta leer el código y puede sobrescribir nombres existentes.
- Es preferible importar el módulo completo o importar recursos concretos de forma explícita.

```

"""
Evitar
from validaciones import *
Mejor
import validaciones
O bien
from validaciones import puerto_valido
"""
# main.py con import * (evitar)

from validaciones import *

puerto = 443
if puerto_valido(puerto):
    print("Puerto correcto")

servicio = normalizar_servicio(" SsH ")
print(servicio)

```

## Módulo importable frente a script ejecutable

- Un fichero puede ser importado como módulo o ejecutado directamente como script.
- La variable especial `__name__` permite distinguir ambos casos.
- Cuando el fichero se ejecuta directamente, `__name__` se asigna al valor `"__main__"`.
- Cuando se importa, `__name__` tiene como valor el nombre del módulo, es decir, el nombre del fichero sin `.py`.

Evitar la ejecución accidental al importar:

- Si un módulo contiene código suelto, ese código puede ejecutarse accidentalmente al importarlo, provocando un efecto no deseado.

- Por eso conviene colocar las pruebas o la ejecución principal dentro de un bloque `if __name__ == "__main__":`.
- Esto evita salidas inesperadas o modificaciones de datos cuando el fichero se usa como módulo.

```
# diagnostico.py
# Bien: las definiciones no se ejecutan
# y podemos usarlas al importar

def comprobar():
    return "sistema operativo OK"

# Mal: este código se ejecutaría al importar
# print("iniciando diagnóstico")

# Bien: este código solo se ejecuta si
# lanzo el script directamente
if __name__ == "__main__":
    print("iniciando diagnóstico")
    print(comprobar())

# -----
# main.py invocando diagnostico.py
import diagnostico

comprobacion = diagnostico.comprobar()
print(comprobacion)
```

## El directorio `__pycache__`

- El directorio `__pycache__` aparece cuando cargamos módulos.
- Almacena la compilación a bytecode, en archivos `.pyc`, de los módulos importados para optimizar el rendimiento del intérprete.
- Esto evita la re-traducción del código fuente en ejecuciones sucesivas, acelerando el tiempo de carga del script de forma automática.
- **Es volátil:** su eliminación no afecta al código y se regenera de forma transparente al detectar cambios.
- En repositorios Git es conveniente excluirlo del control de versiones mediante `.gitignore`.

```
[user@station1 Tema09]$ ls
diagnostico.py
main_alias_completo.py
main_alias_recurso.py
main_alias_todo.py
main_completo.py
main_diagnostico.py
main_recursos.py
__pycache__
validaciones.py

[user@station1 Tema09]$ tree __pycache__/
__pycache__/
├── diagnostico.cpython-313.pyc
└── validaciones.cpython-313.pyc
```

## Organización de un proyecto pequeño

- La convención habitual para organizar pequeños proyectos es dividirlo en varios ficheros dentro de un mismo directorio.
- El script principal, normalmente `main.py`, debe coordinar el flujo.
- Los módulos auxiliares deben contener funciones o clases reutilizables.
- Una estructura sencilla evita mezclar lectura, validación, lógica y presentación en el mismo fichero.

### Cómo busca Python los módulos:

- Python busca módulos en varias ubicaciones. La primera suele ser el directorio desde el que se ejecuta el script.
- Después revisa rutas configuradas en el entorno y en la instalación de Python.
- Es importante evitar nombres como `random.py`, `json.py`, `pathlib.py` o `sys.py` porque pueden ocultar módulos reales que estén más abajo en la lista de búsqueda.
- `sys.path` permite ver esas rutas, aunque no conviene modificarlo sin necesidad.

```
# Tema09/
# └─ main.py
# └─ inventario.py
# └─ validaciones.py
# └─ reportes.py
#
# main.py importa y coordina
# Si el módulo está junto a main.py,
# normalmente se podrá importar.

import sys

for ruta in sys.path[:5]:
    print(ruta)
```

## Organización en paquetes

- Un paquete es un directorio que agrupa módulos relacionados.
- En muchos proyectos se incluye un fichero `__init__.py`. Desde Python 3.3 ya no es obligatorio, pero sigue siendo útil para marcar el directorio como paquete y controlar inicializaciones o exportaciones.
- Los paquetes permiten ordenar código por áreas: inventario, red, reportes o configuración.

### Imports absolutos y relativos:

- Un import absoluto usa el nombre completo desde la raíz del paquete.
- Un import relativo usa puntos para referirse a módulos del mismo paquete.

```
# Tema09/
# └─ main.py           <--- Principal ejecutable
# └─ soporte_ti/       <--- Paquete
#     └─ __init__.py    <--- Control del paquete
#     └─ red.py         <--- Módulos
#     └─ ficheros.py
#     └─ reportes.py

# Import absoluto desde main.py
from soporte_ti import red
from soporte_ti.red import validar_ip
```

```
# Import relativo desde un módulo hermano
# dentro del paquete, por ejemplo de ficheros.py a red.py
from .red import validar_ip

# En proyectos simples, dentro del mismo directorio
import validaciones
```

## Diseño y buenas prácticas con módulos propios

- Agrupar funciones, clases o constantes relacionadas.
- Mantener módulos pequeños y con una responsabilidad clara.
- Separar lógica reutilizable de código de ejecución.
- Usar nombres claros: `validaciones.py`, `inventario.py`, `reportes.py`.
- Documentar el propósito del módulo con un docstring inicial.
- **Evitar efectos laterales al importar:** no leer ficheros, no pedir `input()`, no ejecutar procesos automáticamente.
- Usar `main()` y `if __name__ == "__main__"` para pruebas o ejecución local.

```
# validaciones.py
"""Utilidades de validación para servicios."""

def puerto_valido(puerto: int) -> bool:
    """Devuelve True si el puerto es válido."""
    return 1 <= puerto <= 65535

def servicio_normalizado(nombre: str) -> str:
    """Normaliza el nombre de un servicio."""
    return nombre.strip().lower()

def main():
    print(puerto_valido(443))
    print(servicio_normalizado("  SSH  "))

if __name__ == "__main__":
    main()
```

## Errores frecuentes al importar módulos

- `ModuleNotFoundError`: Python no encuentra el módulo.
- `ImportError`: el módulo existe, pero el recurso importado no.
- Nombres de fichero que colisionan con módulos estándar: `random.py`, `json.py`, `sys.py`.
- Usar `from modulo import *` puede provocar colisiones con nombres ya existentes en el código.
- Ejecutar el script desde un directorio distinto al esperado.
- Código que se ejecuta accidentalmente al importar.
- Importaciones circulares: dos módulos se importan entre sí.

```
# Error habitual: el recurso
# no existe en validaciones.py
from validaciones import validar_ip

# Diagnóstico rápido
import sys
print(sys.path)
```

## Resumen del tema 9

- Un módulo es un fichero `.py` que agrupa código reutilizable: funciones, clases, constantes y variables.
- `import`, `from ... import` y `as` permiten usar módulos completos, recursos concretos o alias.
- El bloque `if __name__ == "__main__"` evita que el código de prueba se ejecute al importar.
- `__pycache__` almacena bytecode generado automáticamente y no debe versionarse.
- Los paquetes organizan módulos en directorios y ayudan a estructurar proyectos más grandes.
- Python localiza módulos en el directorio de ejecución y en `sys.path`. Conviene evitar nombres que colisionen con otros módulos.
- Idea clave: modularizar no busca “más ficheros”, sino separar responsabilidades para que el código sea legible, reutilizable y fácil de diagnosticar.

```
from soporte_ti import red
from soporte_ti import reportes

def main():
    resultado = red.comprobar()
    print(reportes.generar_resumen(resultado))

if __name__ == "__main__":
    main()
```

## Flujo de trabajo en el laboratorio

- Desde el repositorio Git podrás cargar el directorio del Tema09.
- Trabajaremos con la creación de módulos sencillos, diferentes formas de importación, ejecución directa con `__name__`, paquetes y diagnóstico de errores de importación.
- La versión de Python usada en el venv del curso será Python 3.13.

```
# Cargar el entorno
cd ~/Curso_Python
source .venv/bin/activate
python --version

# Actualizar del repositorio Git
git pull origin main

# Para ejecutar VS Code
code . # usar carpeta Tema09

# Para ejecutar en terminal
python
```